Problem 11: LLM Red-Teaming, Guardrail, and Safety Evaluation Platform
Domain: Generative AI | AI Safety | LLMOps | NLP
Difficulty: Hard
Status: NEW (added to reach 15)

Problem

Teams ship LLM applications without systematically testing them for prompt injection, jailbreaks, data leakage, toxicity, bias or hallucination. Manual testing is inconsistent, and guardrails are often added without measuring what they block or break.

Objective

Build a platform that automatically red-teams an LLM application, applies configurable guardrails, and reports measurable risk before and after mitigation.

Users

AI engineers, product security teams, compliance reviewers.

Core Requirements

- Connect to a target LLM app (API or local model) as a black box.
- Attack library: jailbreaks, prompt injection (direct and via retrieved documents), system-prompt extraction, PII leakage probes.
- Automatic attack generation/mutation (templates + LLM-generated variants).
- Automatic judging of responses (classifier or LLM-judge, validated against human labels).
- Input and output guardrails (policy classifier, PII redaction, injection detector).
- Risk report by category and severity.
- Before/after comparison with guardrails on.
- Measure over-refusal on benign prompts.
- Regression test suite that runs on every model/prompt change.
- Dashboard with examples, scores, and reproducible runs.

Advanced Challenges

- Multi-turn and indirect injection attacks (RAG poisoning).
- Judge reliability: agreement with human labels, judge bias.
- Adaptive attacker loop (attacker LLM refines prompts using feedback).
- Multilingual attacks (including Hindi / code-mixed).
- Trade-off curve: attack success vs over-refusal vs latency.
- Detecting guardrail bypass by encoding or obfuscation.

Suggested Datasets/Resources

HarmBench, AdvBench, JailbreakBench, XSTest (over-refusal), ToxiGen, BBQ (bias), TruthfulQA, Llama Guard, NeMo Guardrails, garak, PyRIT.

Suggested Tech

Python, FastAPI, Hugging Face, vLLM/Ollama for local targets, LangGraph for attacker agent, Presidio for PII, Streamlit/React.

Reference Architecture

Attack generator → target app adapter → response judge → scorer/aggregator → guardrail layer (pre/post) → report builder → CI hook.

Evaluation

Attack success rate (ASR) by category, false-refusal rate, judge-human agreement (Cohen's kappa), guardrail latency overhead, coverage of OWASP LLM Top 10 categories.

Safety Note

Use only on models/apps you own or are authorized to test. Do not publish working harmful outputs; store and display redacted examples.

Pitfalls

Judging with the same model that is attacked; counting refusal as safe without checking over-refusal.

Suggested Timeline (48h)

0-8h target adapter + attack set; 8-20h judge + validation; 20-32h guardrails; 32-40h adaptive attacker; 40-48h dashboard + CI.

Standard Judging Rubric (applies to every problem)

Problem understanding	10%
Technical depth	20%
Data engineering (quality, leakage prevention, reproducibility)	10%
Model performance (metrics, baselines, honest evaluation)	15%
System architecture	15%
Working prototype	10%
Explainability and responsible AI	10%
Innovation and scalability	5%
Presentation and demo	5%

Mandatory Submission Checklist

- □ GitHub repository with setup instructions
- □ Working prototype or recorded demo
- □ Documented system architecture
- □ Dataset sources and preprocessing methodology
- □ Evaluation results with baseline comparisons
- □ Limitations and failure cases
- □ One technically challenging feature demonstrated
- □ Short presentation: problem, approach, results, future work